ZX Bank — Conversational AI Backend + Chat UI

> I built a production-grade conversational AI system for ZX Bank that answers customer queries using **Hybrid RAG** (BM25 + FAISS), **multi-turn conversation memory**, **adversarial safety**, and **human escalation** — powered by **GPT-4.1-nano** via **Requesty**, with **auto-detected GPU/CPU** local processing.

📋 Table of Contents

- Architecture
- Tech Stack
- GPU / CPU — Auto Detection
- AI/ML Models & Methods
- LLM Provider — Why Requesty?
- Model Selection — Why GPT-4.1-nano?
- Project Structure
- Setup & Installation
- Running the Application
- API Endpoints
- Retrieval Strategy
- Query Classification & Safety
- Human Escalation Workflow
- Sample Queries for Testing
- Test Dataset
- Observability & Logging

🏗 Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                          User / Chat UI                          │
│                      (SSE Streaming, Port 8000)                  │
└──────────────────────────────────────────────────────────────────┘
                  │
                  │ POST /api/chat/stream (Server-Sent Events)
                  ▼
┌──────────────────────────────────────────────────────────────────┐
│                       FastAPI Server                             │
│                                                                  │
│  ┌──────────────┐   ┌────────────────┐   ┌────────────────────────┐   │
│  │   Session    │   │    Query       │   │   Adversarial          │   │
│  │  Manager     │──▶│  Classifier    │──▶│   Safety Filter        │   │
│  └──────────────┘   └────────────────┘   └────────────────────────┘   │
│                              │                                       │
│                              │                                       │
│           ┌──────────────────┼──────────────────┐                   │
│           ▼                  ▼                  ▼                    │
│  ┌──────────────┐  ┌────────────────┐  ┌────────────────┐            │
```

```
│  │ Small Talk    │ │  Document     │ │  Escalation    │            │
│  │ Handler       │ │  Handler      │ │  Handler       │            │
│  │ (LLM stream)  │ │ (RAG+stream)  │ │ (Stateful)     │            │
│  └───────────────┘ └───────┬───────┘ └────────────────┘            │
│                            │                                        │
│              ┌─────────────────────┐                                │
│              │   Hybrid Retriever  │                                │
│              │  BM25 + FAISS + RRF │                                │
│              └─────────────────────┘                                │
│                            │                                        │
│          ┌─────────────────┼─────────────────┐                    │
│          ▼                 ▼                 ▼                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐             │
│  │ BM25 Index   │  │ FAISS Index  │  │ SBERT        │             │
│  │  (Sparse)    │  │  (Dense)     │  │ (Embeddings) │             │
│  └──────────────┘  └──────────────┘  └──────────────┘             │
│                                                                     │
│              ┌─────────────────────┐                                │
│              │   Requesty API      │                                │
│              │ (GPT-4.1-nano)      │                                │
│              │ SSE Token Streaming │                                │
│              └─────────────────────┘                                │
└─────────────────────────────────────────────────────────────────────┘
```

🛠️ Tech Stack

Component	Technology	Purpose
Runtime	Python 3.10+	Core runtime
API	FastAPI + Uvicorn	Async HTTP server with SSE streaming
LLM	GPT-4.1-nano (via Requesty)	Response generation (streamed)
Embeddings	Sentence-Transformers (all-MiniLM-L6-v2)	Dense vector embeddings (GPU/CPU auto-detected)
Vector Search	FAISS (IndexFlatIP)	Dense semantic retrieval
Sparse Search	BM25 (Okapi)	Keyword-based retrieval
Keyword Extraction	TF-IDF (scikit-learn)	Automated keyword extraction
Re-Ranking	Reciprocal Rank Fusion (RRF)	Hybrid result fusion
Doc Splitting	LangChain MarkdownHeaderTextSplitter	Structure-preserving chunking
Config	Pydantic Settings + .env	Type-safe configuration
Logging	structlog + rich	Structured observability

Component	Technology	Purpose
Frontend	HTML + CSS + JS	Premium dark-mode chat UI with SSE

⚡ GPU / CPU — Auto Detection

I built the system to **automatically detect** whether a GPU is available and fall back to CPU if not. No configuration needed — it just works on any machine.

Component	GPU Available	No GPU	Why
SBERT Embeddings	Runs on **CUDA GPU**	Runs on **CPU**	Auto-detected via PyTorch. On CPU, query encoding takes ~15ms (vs ~5ms on GPU) — no noticeable difference.
TF-IDF	CPU	CPU	Sparse matrix operation — CPU is optimal.
BM25	CPU	CPU	Simple scoring over inverted indices. Sub-millisecond.
FAISS	CPU	CPU	With 662 vectors at 384-dim, CPU takes <2ms. GPU FAISS only helps at 100K+ vectors.
LLM (GPT-4.1-nano)	**Remote API**	**Remote API**	Runs on OpenAI's servers via Requesty. Not affected by local hardware.

> **Bottom line**: You do **NOT** need a GPU to run this system. The pre-built indexes are included in the repo. The system runs identically on CPU — embedding a single query takes ~15ms on CPU which is invisible since the LLM API call takes 500-2000ms anyway.

I developed this on an **NVIDIA RTX 4050 (6GB VRAM, CUDA)** but tested and ensured it works perfectly on CPU too.

🤖 AI/ML Models & Methods

Local Models (run on your machine)

Model / Method	Type	Parameters	What It Does	Runs On
all-MiniLM-L6-v2	Sentence-BERT (Transformer)	22.7M	Converts text → 384-dimensional dense vectors for semantic similarity search	**GPU or CPU** (auto)
TF-IDF (TfidfVectorizer)	Statistical NLP	N/A	Extracts top-10 keywords per document chunk using bigram TF-IDF scoring. Used for metadata enrichment.	CPU

Model / Method	Type	Parameters	What It Does	Runs On
BM25 (Okapi BM25)	Probabilistic IR	N/A	Ranks documents by keyword relevance using term frequency, inverse document frequency, and document length normalization.	CPU
FAISS (IndexFlatIP)	Vector Search	N/A	Brute-force cosine similarity search over normalized embeddings. Returns top-K nearest neighbors.	CPU
Reciprocal Rank Fusion	Ensemble Method	N/A	Combines BM25 (sparse) and FAISS (dense) rankings into a unified score using `RRF(d) = Σ 1/(k + rank(d))` where k=60.	CPU

Remote Model (API call)

Model	Provider	What It Does	Latency
GPT-4.1-nano	OpenAI via Requesty	Generates natural language responses from retrieved context. Streamed token-by-token via SSE.	~0.5-1s first token

> **Important**: No model training happens anywhere in this system. All models are **pre-trained**. I only perform **inference** (forward pass) on the local SBERT model and make API calls to the remote LLM.

🔗 LLM Provider — Why Requesty?

> **Note**: The assignment suggested using OpenRouter as the LLM provider. However, I encountered **payment/card processing issues** with OpenRouter's platform that prevented me from using it.

I chose **Requesty** (`router.requesty.ai`) as the LLM router instead. Requesty offers:

- ☑ **Near-zero downtime** for all models with automatic failover
- ☑ **Great latency** with intelligent routing
- ☑ **300+ models** including OpenAI, Anthropic, Google, Mistral
- ☑ **OpenAI SDK compatible** — uses the standard `openai` Python package
- ☑ **SSE streaming support** for real-time token delivery

Provider-Agnostic Design

The system is **fully provider-agnostic**. Switching to OpenRouter or any other OpenAI-compatible endpoint requires only changing two environment variables:

```
LLM_BASE_URL=https://openrouter.ai/api/v1   # or any other provider
REQUESTY_API_KEY=your_api_key_here
```

No code changes needed.

Model Selection — Why GPT-4.1-nano?

For the banking assistant, I needed the **fastest and most cost-effective** model that delivers high-quality grounded responses:

Model	First Token Latency	Cost (Input / Output per 1M)	Verdict
`google/gemini-3-flash-preview`	30-120+ seconds	$0.15 / $0.60	❌ Preview model — unreliable latency
`openai/gpt-4.1`	3-5 seconds	$2.00 / $8.00	❌ Overkill, too expensive
`openai/gpt-4.1-mini`	~1-2 seconds	$0.40 / $1.60	⚠️ Fast but costlier than needed
`openai/gpt-4.1-nano`	**~0.5-1 second**	**$0.10 / $0.40**	☑ **Fastest & cheapest — ideal for RAG**

I went with **GPT-4.1-nano** because the response quality is driven by the **retrieval pipeline**, not the LLM's raw reasoning. The LLM's job is to synthesize well-retrieved context — nano handles this perfectly at 20x less cost than GPT-4.1.

Project Structure

```
├── app/                          # Application source code
│   ├── main.py                   # FastAPI app with lifespan hooks
│   ├── config.py                 # Pydantic settings (auto GPU/CPU detection)
│   ├── api/
│   │   ├── routes.py             # HTTP endpoints + SSE streaming
│   │   └── schemas.py            # Request/response Pydantic models
│   ├── core/
│   │   ├── classifier.py         # Query type classification
│   │   ├── conversation.py       # Multi-turn memory (sliding window)
│   │   ├── llm.py                # LLM client (sync + streaming)
│   │   └── safety.py             # Adversarial detection
│   ├── retrieval/
│   │   ├── document_processor.py # Markdown loading + splitting
│   │   ├── embeddings.py         # SBERT embeddings (GPU/CPU auto)
│   │   ├── bm25_index.py         # BM25 sparse index
│   │   ├── tfidf_extractor.py    # TF-IDF keyword extraction
│   │   ├── vector_store.py       # FAISS dense index
│   │   └── hybrid_retriever.py   # BM25 + FAISS + RRF fusion
│   ├── handlers/
│   │   ├── document_handler.py   # RAG-grounded answers
│   │   ├── smalltalk_handler.py  # Casual conversation
│   │   └── escalation_handler.py # Human handoff (stateful)
│   └── utils/
│       ├── logger.py             # structlog + rich setup
│       └── helpers.py            # Shared utilities
```

```
├── data/
│   ├── documents/                # 71 ZX Bank knowledge base documents
│   └── escalations/              # Stored escalation records (JSON)
├── indexes/                      # Pre-built FAISS + BM25 indexes (662 chunks)
├── frontend/                     # Chat UI (HTML + CSS + JS, SSE)
├── queries/
│   ├── standard_queries.json     # 20 curated test queries
│   ├── adversarial_queries.json  # Adversarial/safety test queries
│   └── test_dataset.csv          # Full 326-row test dataset (from company)
├── .env.example                  # Environment template
├── .gitignore                    # Git ignore rules
├── requirements.txt              # Python dependencies (flexible versions)
├── run.py                        # One-command startup script
├── setup_index.py                # Index building script
└── README.md                     # This file
```

🚀 Setup & Installation

Prerequisites

- **Python 3.10+** (tested on 3.10, should work on 3.11/3.12 too)
- Requesty API key (get one at requesty.ai)
- **GPU is optional** — system auto-detects and works on CPU

Step 1: Install Dependencies

```
pip install -r requirements.txt
```

> **PyTorch Note**: If `torch` is not already installed, install it separately:
>
> - **CPU only**: `pip install torch --index-url https://download.pytorch.org/whl/cpu`
> - **With CUDA (NVIDIA GPU)**: `pip install torch --index-url https://download.pytorch.org/whl/cu121`

Step 2: Configure Environment

```
cp .env.example .env
# Edit .env and set your Requesty API key
```

Only **one required setting** in `.env`:

```
REQUESTY_API_KEY=your_key_here          # Required — get from requesty.ai
LLM_MODEL=openai/gpt-4.1-nano          # Optional (this is the default)
LLM_BASE_URL=https://router.requesty.ai/v1  # Optional (this is the default)
```

> **For Evaluators**: The API key is shared separately for security. Just paste it into `.env`. The `.env` file is excluded from Git — this is a security best practice.

Step 3: Indexes (Pre-Built — No Action Needed)

The indexes are **already built and included** in the `indexes/` folder. You don't need to run anything.

If you want to rebuild them (optional):

```
python setup_index.py
```

This processes all 71 markdown documents → 662 chunks with SBERT embeddings + BM25 index.

▶️ Running the Application

Single command:

```
python run.py
```

This automatically:

1. ☑ Kills any existing process on port 8000
2. ☑ Detects GPU/CPU and displays it
3. ☑ Checks for indexes (builds if missing)
4. ☑ Sets HuggingFace offline mode (no model re-downloads)
5. ☑ Starts the FastAPI server

Open **http://localhost:8000** for the chat UI with **real-time SSE streaming**.

> **First Run Note**: The SBERT model (`all-MiniLM-L6-v2`, ~80MB) downloads once on first run and gets cached locally. All subsequent runs load from cache with zero network access.

📡 API Endpoints

Method	Endpoint	Description
`POST`	`/api/chat/stream`	**Primary** — SSE streaming (word-by-word)
`POST`	`/api/chat`	Full response (non-streaming)
`GET`	`/api/health`	System health check
`GET`	`/api/history/{session_id}`	Conversation history
`GET`	`/api/escalations`	List escalation records

POST /api/chat/stream (SSE)

Streams tokens in real-time via Server-Sent Events:

```
event: meta
data:
{"session_id":"abc123","query_type":"document_query","confidence":0.95,"sources":
[...]}

event: token
data: "ZX"

event: token
data: " Bank"

event: done
data: {}
```

🔍 Retrieval Strategy

Hybrid RAG Pipeline

1. **BM25 (Sparse)**: Keyword-based matching using Okapi BM25. Great for exact terms like "CIBIL score 700", interest rates, or specific branch names.
2. **FAISS (Dense)**: Semantic search using SBERT embeddings. Great for meaning-based matching — e.g., "how to get money quickly" → finds Gold Loan info.
3. **Reciprocal Rank Fusion (RRF)**: `RRF(d) = Σ 1/(k + rank(d))` where `k=60`. Produces a unified ranking that leverages both sparse and dense strengths.

Document Processing

- **71 real ZX Bank documents** covering: accounts, loans (personal/car/bike/gold/house/agriculture/business), credit cards, UPI, NetBanking, mobile app, bill payments, cross-border payments, safety/security, ATM locations (hospitals, malls, theaters, parks, restaurants, petrol pumps, colleges, tech parks, railway stations, beach roads), branch networks (20+ Indian cities + Sri Lanka, Bangladesh, Nepal, Bhutan), insurance, locker, cheque book, and more.
- **662 chunks** with metadata: `doc_title`, `section_heading`, `doc_type`, and TF-IDF `keywords`
- **Splitter**: `MarkdownHeaderTextSplitter` preserves heading hierarchy
- **Embeddings**: 384-dim L2-normalised vectors from all-MiniLM-L6-v2

🛡 Query Classification & Safety

Type	Trigger	Handler	Retrieval?
`document_query`	Banking questions	Hybrid RAG + LLM streaming	☑ Yes
`small_talk`	Greetings, thanks	LLM streaming (no retrieval)	❌ No
`escalation`	"speak to agent"	Stateful collection flow	❌ No

Type	Trigger	Handler	Retrieval?
`adversarial`	Jailbreak, injection	Instant safe refusal	❌ No

Multi-Layer Adversarial Safety

1. **Pattern Matching**: Regex detection for prompt injection, jailbreak, social engineering
2. **Deep Safety Check**: Secondary analysis on all incoming document queries
3. **Contextual Refusals**: Different refusal messages per attack type

---

🤝 Human Escalation Workflow

```
User: "I want to speak to a human"
Bot:  "May I have your full name?"          ← Step 1
User: "Rahul Sharma"
Bot:  "Could you share your phone number?"  ← Step 2
User: "+91-9876543210"
Bot:  "What's the reason for your request?" ← Step 3
User: "Loan stuck for 2 weeks"
Bot:  "✅ Recorded! Reference: C8D3..."      ← Saved to JSON
```

- Validated phone number format (7-15 digits)
- Persisted to `data/escalations/` as JSON
- Retrievable via `GET /api/escalations`

💬 Sample Queries for Testing

I included 20 curated queries in `queries/standard_queries.json` covering all 7 query types from the test dataset:

#	Query	Type
Q01	What are the features of ZX Bank's Savings Account?	Descriptive
Q02	How do I apply for a car loan at ZX Bank?	Procedural
Q03	What is the interest rate for ZX Bank's Gold Loan?	Descriptive
Q04	Does ZX Bank have ATMs at hospitals in Mumbai?	Boolean
Q05	How do I report a fraud transaction?	Procedural
Q06	What types of credit cards does ZX Bank offer?	Descriptive
Q07	How does the branch network in Mumbai compare to Delhi?	Comparative
Q08	What is UPI and how do I activate it?	Procedural
Q09	When did ZX Bank receive the Best Digital Transformation award?	Temporal

#	Query	Type
Q10	Why might ZX Bank place ATMs near movie theaters?	Analytical
Q11	What are the features of ZX Bank's Agriculture Loan?	Descriptive
Q12	Is the ZX Bank Bike Loan available for electric bikes?	Boolean
Q13	What safety features does ZX Bank have?	Descriptive
Q14	How do I convert my salary account to a savings account?	Procedural
Q15	What types of business loans does ZX Bank offer?	Open-Ended
Q16	Does ZX Bank offer cross-border payment services?	Boolean
Q17	How do I open a locker at ZX Bank?	Procedural
Q18	What is the Personal Relationship Manager program?	Descriptive
Q19	What is the maximum house loan amount?	Descriptive
Q20	What can I do with ASK Zia, the ZX Bank chatbot?	Open-Ended

Adversarial Queries

#	Attack Type	Expected Behaviour
A01	Prompt Injection	Safe refusal
A02	Jailbreak (DAN)	Safe refusal + redirect
A03	Social Engineering	Contextual refusal

📊 Test Dataset

The full test dataset provided by the company is included at `queries/test_dataset.csv`. It contains **326 queries** across 7 query types:

Query Type	Count	Description
Procedural	~50	"How do I..." step-by-step questions
Descriptive	~60	"What are the features of..." questions
Boolean	~45	"Does ZX Bank offer..." yes/no questions
Comparative	~35	"How does X compare to Y" questions
Analytical	~40	"Why might..." reasoning questions
Open-Ended	~50	Broad questions requiring comprehensive answers
Temporal	~46	"When did..." time-based questions

Each row includes the query, query type, expected answer, supporting facts, and source filenames.

📊 Observability & Logging

Every request produces structured terminal logs via `structlog`:

```
2026-03-05 14:30:00 [info] query_classified   query_type=document_query
session_id=a1b2c3
2026-03-05 14:30:00 [info] retrieval_timing   elapsed_seconds=0.005 results=5
2026-03-05 14:30:02 [info] llm_stream_complete first_token_seconds=1.23
tokens_streamed=145
```

Log Field	Description
`query_type`	Classification result
`retrieval_triggered`	Whether RAG pipeline was used
`elapsed_seconds`	Retrieval latency
`first_token_seconds`	Time to first LLM token
`tokens_streamed`	Number of tokens generated
`session_id`	Conversation session ID

📄 License

This project was built as part of the **ML Intern Assignment** for Yellow.ai.